

Task 1 - Report

24520015 - Trịnh Duy Hưng

Ngày 1 tháng 6 năm 2026

Tóm tắt nội dung

Báo cáo này trình bày quy trình xây dựng hệ thống gợi ý sản phẩm cho bài toán recommendation. Mục tiêu của hệ thống là dự đoán danh sách 10 sản phẩm phù hợp nhất cho mỗi khách hàng. Dữ liệu đầu vào ban đầu được chuẩn hóa thành một bảng lịch sử tương tác thống nhất gồm các cột `customer_id`, `item_id`, `price`, `quantity`, `event_type`, `updated_date` và `discount`. Hành vi chính được sử dụng là `purchase`. Hệ thống được xây dựng theo hướng two-stage recommender: đầu tiên sinh tập ứng viên bằng top-popular items và lịch sử mua hàng của từng user, sau đó sử dụng LightGBM với objective `lambdarank` để xếp hạng các item và lấy ra top 10 item cho mỗi khách hàng.

1 Giới thiệu bài toán

Bài toán đặt ra là xây dựng một hệ thống gợi ý sản phẩm cho khách hàng dựa trên lịch sử mua hàng. Với mỗi khách hàng, hệ thống cần trả về một danh sách gồm 10 sản phẩm được dự đoán là có khả năng phù hợp nhất.

Đầu vào của hệ thống là:

- `customer_id`: mã khách hàng;
- lịch sử tương tác/mua hàng trong quá khứ;
- thông tin sản phẩm như giá, số lượng, giảm giá và thời điểm giao dịch.

Đầu ra yêu cầu là một file `.pkl` có dạng:

```
{  
    customer_id: [pred_item1, pred_item2, ..., pred_item10]  
}
```

Trong đó, mỗi `customer_id` được ánh xạ tới danh sách 10 `item_id` được recommend.

2 Metric đánh giá

Metric được sử dụng là **Precision@10**. Với mỗi user, hệ thống lấy 10 item đầu tiên trong danh sách dự đoán, sau đó đếm số item nằm trong groundtruth.

Công thức Precision@10 cho một user:

$$Precision@10 = \frac{\# \text{ item dự đoán đúng trong top 10}}{10}$$

Nếu gọi:

- P_u^{10} là tập 10 item được dự đoán cho user u ;
- T_u là tập item thật sự đúng trong groundtruth của user u ;

thì:

$$Precision@10(u) = \frac{|P_u^{10} \cap T_u|}{10}$$

Metric cuối cùng là trung bình Precision@10 trên toàn bộ user:

$$MeanPrecision@10 = \frac{1}{|U|} \sum_{u \in U} Precision@10(u)$$

Trong code đánh giá, kết quả cuối cùng được nhân với 100:

$$Score = MeanPrecision@10 \times 100$$

3 Dữ liệu

Ban đầu dữ liệu có thể đến từ nhiều nguồn, ví dụ bảng **transaction** và bảng **event**. Tuy nhiên, để dễ xử lý, các bảng được chuẩn hóa về cùng một format và gộp lại thành một bảng chung.

Bảng cuối cùng có dạng:

Cột	Ý nghĩa
<code>customer_id</code>	Mã khách hàng
<code>item_id</code>	Mã sản phẩm
<code>price</code>	Giá sản phẩm
<code>quantity</code>	Số lượng mua
<code>event_type</code>	Loại hành động, trong phiên bản cuối chủ yếu là purchase
<code>updated_date</code>	Thời điểm phát sinh hành động
<code>discount</code>	Giá trị giảm giá

Bảng 1: Cấu trúc bảng dữ liệu sau khi chuẩn hóa

Trong phiên bản cuối, hệ thống tập trung vào hành vi **purchase**. Điều này có nghĩa là mỗi dòng dữ liệu thể hiện việc một khách hàng đã mua một sản phẩm tại một thời điểm cụ thể.

4 Tiền xử lý dữ liệu

Trước khi xây dựng mô hình, dữ liệu được chuẩn hóa như sau:

- Chuyển `event_type` về chữ thường;
- Ép kiểu `price` sang `Float32`;
- Điền giá trị thiếu cho `quantity` bằng 1;
- Điền giá trị thiếu cho `discount` bằng 0;
- Chuyển `quantity` sang kiểu số nguyên.

Ví dụ:

```
df = (
    df
    .with_columns([
        pl.col("event_type").str.to_lowercase().alias("event_type"),
        pl.col("price").cast(pl.Float32),
        pl.col("quantity").fill_null(1).cast(pl.Int32),
        pl.col("discount").fill_null(0).cast(pl.Float32),
    ])
)
```

Nếu chỉ còn hành động `purchase`, ta có thể xem mọi dòng dữ liệu là một tín hiệu mua hàng. Do đó, `event_weight` có thể được gán bằng 1 hoặc dựa theo số lượng/giá trị giao dịch. Trong báo cáo này, ta ưu tiên sử dụng trực tiếp các thống kê từ hành vi `purchase` như số lần mua, tổng số lượng mua và tổng doanh thu.

5 Chia dữ liệu theo thời gian

Vì recommendation là bài toán có yếu tố thời gian, dữ liệu không được chia ngẫu nhiên. Nếu chia ngẫu nhiên, thông tin tương lai có thể bị đưa vào quá trình train, gây ra hiện tượng data leakage.

Mục tiêu cuối cùng là predict cho tháng 1 năm sau. Do đó, pipeline được thiết kế như sau:

Giai đoạn	Khoảng thời gian	Vai trò
history_train	Trước 01/12/2025	Tạo feature để train
label_dec	01/12/2025 – 31/12/2025	Tạo label train
history_jan	Trước 01/01/2026	Tạo feature để predict tháng 1

Bảng 2: Chiến lược chia dữ liệu theo thời gian

Cụ thể:

```

history_train = df.filter(
    pl.col("updated_date") < datetime(2025, 12, 1)
)

label_dec = df.filter(
    (pl.col("updated_date") >= datetime(2025, 12, 1)) &
    (pl.col("updated_date") < datetime(2026, 1, 1))
)

history_jan = df.filter(
    pl.col("updated_date") < datetime(2026, 1, 1)
)

```

Ý tưởng là mô hình học từ lịch sử trước tháng 12 để dự đoán hành vi mua trong tháng 12. Sau khi mô hình đã học được quy luật ranking, ta dùng toàn bộ lịch sử trước tháng 1 để predict cho tháng 1.

6 Tạo nhãn huấn luyện

Vì bài toán hướng tới dự đoán item mà user sẽ mua, ta dùng các cặp `customer_id`, `item_id` có hành vi purchase trong tháng 12 làm positive label.

```

dec_positive = (
    label_dec
    .filter(pl.col("event_type") == "purchase")
    .select(["customer_id", "item_id"])
    .unique()
    .with_columns(
        pl.lit(1).cast(pl.Int8).alias("label")
    )
)

```

Ý nghĩa:

- Nếu user mua item trong tháng 12, cặp (*user*, *item*) có label bằng 1;
- Các item candidate còn lại không xuất hiện trong danh sách mua sẽ có label bằng 0.

Ví dụ:

customer_id	item_id	label
A	X	1
A	Y	0
A	Z	0

Bảng 3: Ví dụ label cho một user

Với LightGBM ranking, mô hình sẽ học cách xếp item có label 1 lên cao hơn item có label 0 trong cùng một group user.

7 Feature Engineering

Mỗi dòng dữ liệu train tương ứng với một cặp `customer_id`, `item_id`. Thay vì đưa trực tiếp ID vào mô hình, ta tạo các feature mô tả user, item và quan hệ giữa user với item.

7.1 User Features

User features mô tả hành vi mua hàng tổng quát của khách hàng.

Các feature chính:

Feature	Ý nghĩa
<code>user_purchase_count</code>	Tổng số lần user mua hàng
<code>user_unique_items</code>	Số item khác nhau user từng mua
<code>user_total_quantity</code>	Tổng số lượng sản phẩm user đã mua
<code>user_total_spending</code>	Tổng số tiền user đã chi
<code>user_avg_price</code>	Giá trung bình các item user đã mua
<code>user_avg_discount</code>	Mức giảm giá trung bình user nhận được
<code>user_recency_days</code>	Số ngày từ lần mua gần nhất của user đến thời điểm dự đoán

Bảng 4: Nhóm user features

Ví dụ công thức:

$$user_total_spending = \sum_{i \in I_u} price_i \times quantity_i$$

Trong đó I_u là tập các giao dịch của user u .

7.2 Item Features

Item features mô tả độ phổ biến và đặc điểm bán hàng của sản phẩm.

Feature	Ý nghĩa
<code>item_purchase_count</code>	Tổng số lần item được mua
<code>item_unique_buyers</code>	Số user khác nhau từng mua item
<code>item_total_quantity</code>	Tổng số lượng item đã bán
<code>item_total_revenue</code>	Tổng doanh thu của item
<code>item_avg_price</code>	Giá trung bình của item
<code>item_avg_discount</code>	Giảm giá trung bình của item
<code>item_recency_days</code>	Số ngày từ lần mua gần nhất của item đến thời điểm dự đoán

Bảng 5: Nhóm item features

Một item có nhiều người mua, doanh thu cao và vẫn được mua gần đây thường là item tốt để recommend.

7.3 User-Item Features

User-item features mô tả mối quan hệ giữa một user cụ thể và một item cụ thể.

Feature	Ý nghĩa
<code>ui_purchase_count</code>	Số lần user từng mua item này
<code>ui_total_quantity</code>	Tổng số lượng user từng mua item này
<code>ui_total_spending</code>	Tổng tiền user đã chi cho item này
<code>ui_avg_price</code>	Giá trung bình user mua item này
<code>ui_avg_discount</code>	Discount trung bình user nhận được với item này
<code>ui_recency_days</code>	Số ngày từ lần mua gần nhất của user với item này
<code>ui_has_purchased</code>	User đã từng mua item này hay chưa

Bảng 6: Nhóm user-item features

Nhóm feature này rất quan trọng vì nó phản ánh sở thích cá nhân của từng user. Nếu user từng mua một item nhiều lần hoặc mua gần đây, item đó có thể có khả năng được mua lại.

7.4 Interaction Features

Sau khi có user features và item features, ta tạo thêm các feature so sánh giữa user và item:

$$price_diff_user_item = item_avg_price - user_avg_price \quad (1)$$

$$price_ratio_user_item = \frac{item_avg_price}{user_avg_price + 1} \quad (2)$$

$$discount_diff_user_item = item_avg_discount - user_avg_discount \quad (3)$$

Các feature này giúp mô hình học được item có phù hợp với thói quen chi tiêu của user hay không.

8 Candidate Generation

Không thể chấm điểm toàn bộ item cho toàn bộ user vì số lượng cặp user-item quá lớn. Do đó, hệ thống sử dụng bước candidate generation để lấy ra một tập item ứng viên nhỏ hơn cho mỗi user.

Pipeline gồm hai nguồn candidate chính:

$$Candidate(u) = TOP_POP \cup HIST_K(u)$$

8.1 TOP_POP

TOP_POP là danh sách các item phổ biến nhất toàn hệ thống. Khi chỉ sử dụng hành vi purchase, TOP_POP có nghĩa là:

Các item được mua nhiều nhất hoặc có điểm phổ biến cao nhất trong lịch sử.

Có thể tính điểm phổ biến như sau:

$$pop_score_i = \alpha \cdot purchase_count_i + \beta \cdot total_quantity_i + \gamma \cdot unique_buyers_i$$

Trong đó:

- $purchase_count_i$: số lần item i được mua;
- $total_quantity_i$: tổng số lượng item i đã bán;
- $unique_buyers_i$: số khách hàng khác nhau từng mua item i .

Trong phiên bản đơn giản, có thể dùng trực tiếp `purchase_count` hoặc tổng số lượng bán ra để sắp xếp item.

Nếu `TOP_POP = 50`, mỗi user sẽ nhận 50 item phổ biến nhất làm candidate.

8.2 HIST_K

HIST_K là danh sách các item gắn với lịch sử mua hàng của từng user. Khi dữ liệu chỉ có `purchase`, HIST_K có nghĩa là:

Các item mà user từng mua nhiều nhất hoặc mua gần đây nhất.

Ví dụ, với mỗi user, ta xếp hạng các item đã từng mua theo `ui_purchase_count` hoặc `ui_total_spending`, sau đó lấy top K .

Nếu `HIST_K = 50`, mỗi user sẽ nhận tối đa 50 item từ lịch sử mua hàng của chính user đó.

8.3 Gộp Candidate

Sau khi lấy candidate từ hai nguồn, ta gộp lại theo cặp `customer_id`, `item_id`. Nếu một item vừa nằm trong `TOP_POP`, vừa nằm trong lịch sử của user, item đó chỉ được giữ một lần.

Đồng thời tạo hai cột đánh dấu nguồn candidate:

- `cand_from_popular`: item đến từ top popular;
- `cand_from_history`: item đến từ lịch sử user.

Hai cột này cũng được đưa vào mô hình làm feature.

9 Mô hình LightGBM Lambdarank

Sau khi tạo candidate và feature, bài toán trở thành bài toán ranking. Mỗi user là một group, các item candidate là các item cần được xếp hạng trong group đó.

Mô hình được sử dụng là LightGBM với objective:

```
objective = "lambdarank"
```

Lý do dùng `lambdarank` là vì metric cuối cùng là Precision@10, tức là quan tâm đến thứ tự các item trong top đầu. Ranking objective phù hợp hơn binary classification vì mô hình học trực tiếp cách xếp item đứng lên cao trong danh sách candidate.

Cấu hình mô hình:

```
params = {
    "objective": "lambdarank",
    "metric": "ndcg",
    "ndcg_eval_at": [10],
    "learning_rate": 0.05,
    "num_leaves": 64,
    "min_data_in_leaf": 100,
    "feature_fraction": 0.8,
    "bagging_fraction": 0.8,
    "bagging_freq": 1,
    "lambda_l2": 1.0,
    "verbosity": -1,
    "seed": 42,
}
```

Trong LightGBM ranking, cần truyền thêm `group`. Mỗi group tương ứng với số candidate của một customer:

```
group = train_pd.groupby("customer_id").size().to_numpy()
```

Điều này giúp LightGBM hiểu rằng các item trong cùng một group là các item candidate của cùng một user.

10 Predict cho tháng 1

Sau khi train mô hình bằng dữ liệu tháng 12, ta predict cho tháng 1. Điểm quan trọng là không dùng lại feature train cũ, mà phải tạo lại feature từ `history_jan`.

Lý do là feature train chỉ được tính từ lịch sử trước 01/12/2025, trong khi predict tháng 1 phải sử dụng toàn bộ lịch sử trước 01/01/2026. Như vậy, các hành vi mua hàng trong tháng 12 sẽ được phản ánh vào feature tháng 1.

Ví dụ:

- Nếu user mua item X trong tháng 12, `ui_purchase_count` của cặp user-item đó phải được cập nhật;
- Nếu item Y bán rất chạy trong tháng 12, `item_purchase_count` và `pop_score` của item đó phải tăng;
- Nếu user vừa mua hàng gần đây, `user_recency_days` phải nhỏ hơn.

Do đó:

$$Feature_{train} \neq Feature_{jan}$$

Mô hình thì có thể dùng lại, nhưng input feature phải được tính lại tại đúng thời điểm predict.

11 Xử lý dữ liệu lớn bằng bucket

Dữ liệu có thể có hàng chục triệu dòng. Nếu tạo candidate và predict cho toàn bộ user cùng lúc, quá trình join feature và convert sang pandas có thể gây nổ RAM.

Để xử lý vấn đề này, ta chia user thành nhiều bucket theo hash của `customer_id`:

$$\text{bucket}(u) = \text{hash}(\text{customer_id}) \bmod N$$

Trong đó N là số bucket, ví dụ $N = 20$.

Với mỗi bucket, hệ thống chỉ xử lý một phần user:

1. Lấy user trong bucket;
2. Tạo popular candidates;
3. Tạo history candidates;
4. Gộp candidate;
5. Join feature tháng 1;
6. Predict score bằng LightGBM;
7. Lấy top 10 item cho mỗi user;
8. Gộp kết quả vào dictionary `submission`.

Cách này giúp giảm lượng dữ liệu cần đưa vào RAM tại mỗi thời điểm.

12 Tạo file submission

Sau khi predict score cho các candidate, với mỗi user ta sắp xếp item theo score giảm dần và lấy top 10 item.

```
top10 = (
    pred_pd
    .sort_values(["customer_id", "score"], ascending=[True, False])
    .drop_duplicates(["customer_id", "item_id"])
    .groupby("customer_id")
    .head(10)
)
```

Sau đó tạo dictionary:

```
submission = {
    customer_id: [item1, item2, ..., item10]
}
```

Nếu một user có ít hơn 10 item candidate, hệ thống dùng các item phổ biến nhất để fill thêm cho đủ 10 item. Cuối cùng lưu file:

```
with open("/kaggle/working/submission_jan.pkl", "wb") as f:
    pickle.dump(submission, f)
```